

Guía de trabajo del equipo — Scaps

Cómo trabajamos y qué hay para hacer en el Sprint 0

Scaps es nuestro e-commerce de gorras con visor de productos en 3D. Esta guía explica, en pocos minutos, cómo nos organizamos día a día y cuáles son las tareas del primer sprint. Léela antes de tomar tu primera tarea.

Cómo nos organizamos

Trabajamos con un tablero de tareas en **GitHub Projects**, no con roles fijos. No hay "dueños" de áreas: el trabajo está en un backlog priorizado y **cada uno toma la tarea que sigue**. Una tarea avanza por estas cinco columnas, de izquierda a derecha:



- **Backlog** — lo que todavía no está listo para tomar (por ejemplo, espera que termine otra tarea de la que depende).
- **To Do** — listo para tomar. Tareas priorizadas que ya se pueden agarrar.
- **In Progress** — lo que alguien está haciendo ahora. Una sola por persona.
- **In Review** — terminado, esperando que otro lo revise (Pull Request).
- **Done** — hecho, revisado y mergeado.

El flujo, paso a paso

Cuando te vas a poner a trabajar:

1. Entrás al tablero (GitHub Projects) y mirás la columna **To Do**.
2. **Tomás la tarea de más arriba**. Están ordenadas por prioridad: no elijas la más fácil del medio.
3. La movés a **In Progress** y la **convertís en issue** (clic en la tarjeta → *Convert to issue*). Le ponés los **labels** que figuran en la tarea (área + prioridad) y **te la asignás**, para que el resto vea que la estás haciendo.
4. **Creás una rama** desde `main` para esa tarea (por ejemplo `feat/scaffold-frontend`).
5. **Programás** y vas guardando tus commits en esa rama.
6. Cuando terminás —y cumple el "*hecho cuando*"— **abrís un Pull Request** y en la descripción escribís `Closes #N` (el número del issue).
7. **Otro del equipo lo revisa** y aprueba. Si pide cambios, los hacés y volvéis a pedir revisión.
8. **Se mergea el PR**. El issue se cierra y la tarjeta **pasa sola a Done**: no tenés que mover nada a mano.

Las reglas (pocas y simples)

- **Una sola tarea en curso por persona**. Terminá lo que empezaste antes de tomar otra.
- **Tomá de arriba hacia abajo** en To Do. Lo prioritario primero.
- **No se pushea directo a `main`**. Todo entra por Pull Request con al menos una aprobación.

- **Si te trabás más de un día**, marcá la tarjeta como bloqueada y pedí ayuda. Una tarea crítica frenada en silencio es lo peor que puede pasar.
- **El que revisa** controla que la tarea cumpla su "*hecho cuando*" y que no rompa nada.

Backlog del Sprint 0

El Sprint 0 es preparación: dejar el proyecto listo para empezar a construir. Cada tarea indica su área entre corchetes, sus **labels** (área + prioridad), qué hacer concretamente, el criterio para darla por terminada (*hecho cuando*) y, cuando aplica, de qué depende.

To Do — se pueden tomar ya (no dependen de nada)

[visor-3d] Conseguir modelos `.glb` de un banco gratuito

visor-3d

prioridad-alta

- **Qué hacer:** bajar 3 o 4 modelos 3D en formato `.glb` de un banco de modelos gratuitos (sitios como Sketchfab o Poly Pizza), para usarlos como productos de demo en el catálogo. Usar modelos con licencia **CC0** (uso libre, sin atribución) y/o **CC BY** (uso libre, pero hay que dar crédito al autor). Si se puede, que parezcan gorras o un producto coherente.
- **Hecho cuando:** los archivos `.glb` están en el repo (o en una carpeta/URL accesible para el equipo) y se pueden usar en el catálogo y en el POC. De cada modelo queda anotado autor, fuente y licencia, para poder acreditar los que sean CC BY.

[setup] Inicializar el monorepo con workspaces

setup

prioridad-alta

- **Qué hacer:** crear la estructura base del repo: una carpeta `apps/` con dos subcarpetas, `apps/web` (frontend) y `apps/api` (backend), y un `package.json` en la raíz. En ese `package.json` raíz, agregar la clave `"workspaces": ["apps/*"]`, que le avisa a npm que las dos apps son parte del mismo repo y se instalan juntas. No usar `packages/shared` ni herramientas de monorepo como Turborepo o Nx.
- **Hecho cuando:** la estructura existe, correr `npm install` desde la raíz instala las dependencias de los dos workspaces, y está mergeado en `main`.

[infra] Proteger la rama main

infra

prioridad-alta

- **Tipo:** configuración en GitHub (no es un PR).
- **Qué hacer:** configurar en GitHub una regla de protección sobre la rama `main` para que nadie pueda hacer push directo y todo cambio entre por Pull Request. Se hace en el repo, en **Settings → Branches → Add branch ruleset** (o *Add rule*), activando "Require a pull request before merging" con al menos 1 aprobación.
- **Hecho cuando:** no se puede hacer push directo a `main`; cualquier cambio requiere un PR con al menos una aprobación.

[setup] Diseñar el modelo de datos (ERD)

setup

prioridad-alta

- **Qué hacer:** diseñar el **ERD** (diagrama entidad-relación): definir las tablas/entidades del MVP, sus campos y cómo se relacionan entre sí. Entidades: usuarios, roles, productos (con marca de "destacado", imágenes y modelo `.glb`), stock, carrito y órdenes. Se puede dibujar en una herramienta como dbdiagram.io o draw.io.
- **Hecho cuando:** hay un ERD (diagrama + descripción de entidades, campos y relaciones) acordado por el equipo y guardado en `docs/`.

[setup] Diseñar los wireframes

setup

prioridad-media

- **Qué hacer:** hacer bocetos simples (sin diseño fino, solo dónde va cada elemento) de las pantallas del MVP: landing (visor 3D del destacado + un **CTA** —botón de llamado a la acción— hacia el catálogo), catálogo en cards, ficha de producto (galería de imágenes + opción Ver en 3D), carrito, checkout (con dirección de envío), Mis direcciones, login y dashboard. Pueden ser en Figma, Excalidraw o incluso a mano y fotografiados.
- **Hecho cuando:** hay wireframes de las pantallas principales accesibles para todo el equipo (en `docs/` o un link compartido).

Backlog — esperan que se libere una dependencia

[setup] Definir el contrato de la API

setup

prioridad-alta

- **Depende de:** Modelo de datos (ERD).
- **Qué hacer:** listar todos los **endpoints** (las direcciones que expone la API) del MVP y, para cada uno, anotar el método (GET, POST, etc.), la ruta, qué recibe (request) y qué devuelve (response). Ejemplo: `GET /products` → devuelve la lista de productos. Cubrir productos, auth, carrito y órdenes. Esto es lo que permite que front y back trabajen en paralelo contra una interfaz acordada.
- **Hecho cuando:** hay un documento con los endpoints y sus formatos, acordado por el equipo y guardado en `docs/`.

[setup] Configurar ESLint + Prettier compartidos

setup

prioridad-alta

- **Depende de:** Inicializar el monorepo.
- **Qué hacer:** configurar **ESLint** (marca errores y malas prácticas en el código) y **Prettier** (formatea el código automáticamente) con reglas compartidas para `web` y `api`, definidas desde la raíz del repo. Agregar los scripts de npm para correrlos (por ejemplo `npm run lint` y `npm run format`).
- **Hecho cuando:** `lint` corre en los dos workspaces, el formateo automático funciona y las reglas están commiteadas.

[setup] Scaffold del frontend

setup

prioridad-alta

- **Depende de:** Inicializar el monorepo.
- **Qué hacer:** crear el proyecto base ("scaffold" = estructura inicial que ya levanta, todavía sin las pantallas reales) del frontend en `apps/web`: una app de **React + Vite + TypeScript** con **Tailwind** ya configurado. Se puede arrancar con el creador de Vite (`npm create vite@latest`).
- **Hecho cuando:** `npm run dev` levanta la app en local y se ve una página inicial con estilos de Tailwind aplicados.

[setup] Scaffold del backend

setup

prioridad-alta

- **Depende de:** Inicializar el monorepo.
- **Qué hacer:** crear el proyecto base del backend en `apps/api`: una app de **NestJS + TypeScript**. Se puede arrancar con el CLI de Nest (`nest new`). Incluir un endpoint de **health check**: un `GET /health` que devuelva 200, que sirve para chequear que el servidor está vivo.
- **Hecho cuando:** el backend levanta en local y `GET /health` responde 200.

[infra] Variables de entorno + `.env.example`

infra

prioridad-media

- **Depende de:** Scaffold del frontend y del backend.
- **Qué hacer:** definir las **variables de entorno** de `web` y `api` (la configuración que no va escrita en el código: URLs, claves, credenciales). Crear un archivo `.env.example` que liste los nombres de esas variables pero **sin los valores reales** (sirve de plantilla para que cada uno arme su propio `.env`). Asegurar que `.env` esté en el `.gitignore` para que nunca se suba al repo.
- **Hecho cuando:** existe `.env.example` con las claves necesarias (sin valores sensibles) y `.env` está ignorado por git.

[documentation] README inicial

documentation

prioridad-media

- **Depende de:** Inicializar el monorepo (idealmente con los scaffolds listos).
- **Qué hacer:** escribir el `README.md` con una descripción corta del proyecto y los pasos para levantarlo en local: requisitos (por ejemplo, la versión de Node), cómo instalar las dependencias y cómo correr `web` y `api`.
- **Hecho cuando:** alguien que clona el repo por primera vez puede levantarlo siguiendo solo el README, sin preguntar nada.

[visor-3d] POC del visor 3D

visor-3d

prioridad-alta

- **Depende de:** Scaffold del frontend + Modelos `.glb` del banco.
- **Qué hacer:** hacer una **POC** (prueba de concepto: algo mínimo para validar que la idea funciona). Con **React Three Fiber + drei** (las librerías para 3D en React), cargar uno de los modelos `.glb` del banco, mostrarlo en pantalla y poder rotarlo con el mouse. Para rotar alcanza con **OrbitControls** (un helper de drei que mueve la cámara con el mouse).
- **Hecho cuando:** en local se ve el modelo `.glb` y se puede rotar con el mouse. Valida que el visor es viable.